

# ERDMS\_Project

## Database Design Documentation

*Electronic Records and Document Management System*

### 1. Overview

The ERDMS\_Project database is the persistent data layer for the Electronic Records and Document Management System. It stores user accounts, folder structures, document metadata, access permissions, and a complete audit trail of system activity.

The schema follows relational database principles with proper normalization to eliminate redundancy and maintain referential integrity. Six interconnected tables cover all system functionality.

### 2. Tables at a Glance

| Table                | Primary Key   | Purpose                                                           |
|----------------------|---------------|-------------------------------------------------------------------|
| Roles                | role_id       | Lookup table for user roles (admin, employees)                    |
| Users                | user_id       | User accounts, credentials, role assignment, and status           |
| Folders              | folder_id     | Hierarchical document containers with permission settings         |
| Documents            | document_id   | File metadata, storage paths, and access counters                 |
| Document Permissions | permission_id | Bridge table for fine-grained user-document access                |
| Logs                 | log_id        | Audit trail of all system actions (login, upload, download, etc.) |

### 3. Table Details

#### 3.1 Roles

Lookup table for the two user roles in the system. A separate table allows future role expansion without code changes.

- role\_id — Primary key
- role\_name — 'admin' or 'employees' (unique, not null)

## 3.2 Users

Central table for all user accounts. Every person accessing the system has a record here.

- `user_id` — Primary key
- `name`, `email` — Full name and login email (email is unique, not null)
- `password` — Stored as a bcrypt hash with salt; never plain text
- `role_id` — FK → Roles; determines system permissions
- `status` — 'active' or 'inactive'; defaults to 'active' (enables deactivation without data loss)
- `created_at` — Auto-generated timestamp

## 3.3 Folders

Organizes documents into a hierarchical tree structure via a self-referential parent key.

- `folder_id` — Primary key
- `folder_name` — Display name (not null)
- `parent_folder_id` — FK → Folders (self); NULL = root level, otherwise a subfolder
- `created_by` — FK → Users
- `permissions` — Access default: public | private | restricted (defaults to 'public')
  - Public — any authenticated user can access
  - Private — uploader + explicitly granted users only
  - Restricted — explicitly granted users only
- `created_at` — Auto-generated timestamp

*Example tree structure:*

```
Company Docs (root) → HR Dept → Employee Contracts
Company Docs (root) → Finance Dept → Reports
```

## 3.4 Documents

Stores metadata for every uploaded file. Binary file content is kept on disk; only the path and metadata are stored here.

- `document_id` — Primary key
- `title` — Must be unique within the same folder
- `description`, `category` — Optional metadata fields
- `file_path` — Server path to the physical file
- `file_size`, `file_type` — Bytes and MIME type for validation and display
- `accessibility` — Optional per-document override of folder permission
- `folder_id` — FK → Folders
- `uploaded_by` — FK → Users
- `download_count` — Integer counter; increments on each successful download; defaults to 0
  - Denormalized for performance — avoids repeated COUNT queries on Logs
- `created_at` — Auto-generated timestamp

### 3.5 Document Permissions

Bridge table enabling fine-grained, per-user access control on individual documents. Resolves the many-to-many relationship between Users and Documents.

- permission\_id — Primary key
- document\_id — FK → Documents
- user\_id — FK → Users
- permission\_type — 'view' or 'download'

*A user may have 'download' on Document X but only 'view' on Document Y. This granularity allows precise access management without redundant records.*

### 3.6 Logs

Immutable audit trail. Every significant system action creates a record here. Retained even after users or documents are deleted.

- log\_id — Primary key
- user\_id — FK → Users (set to NULL if user is deleted)
- action — One of: login | logout | upload | download | edit | delete
- document\_id — FK → Documents (nullable; NULL when action is not document-related)
- timestamp — Exact date-time of the action

---

## 4. Relationships

| From              | To                  | Type & Note                                         |
|-------------------|---------------------|-----------------------------------------------------|
| Roles             | Users               | One-to-Many — one role, many users                  |
| Users             | Folders             | One-to-Many — one user creates many folders         |
| Users             | Documents           | One-to-Many — one user uploads many documents       |
| Users             | Logs                | One-to-Many — one user generates many log entries   |
| Folders           | Documents           | One-to-Many — one folder holds many documents       |
| Folders           | Folders (self)      | Self-Referential — parent-child subfolder hierarchy |
| Documents         | Logs                | One-to-Many — one document has many log entries     |
| Users ↔ Documents | via Doc Permissions | Many-to-Many — bridge table with view/download type |

## 5. Data Integrity Rules

| Rule Type      | Field / Table                | Constraint                                              |
|----------------|------------------------------|---------------------------------------------------------|
| Unique         | Users.email                  | No duplicate email addresses                            |
| Unique         | Roles.role_name              | No duplicate role names                                 |
| Unique         | Documents.title (per folder) | Titles unique within same folder                        |
| Not Null       | Users                        | name, email, password, role, status required            |
| Not Null       | Folders                      | folder_name, created_by, permissions required           |
| Not Null       | Documents                    | title, file_path, folder_id, uploaded_by required       |
| Not Null       | Logs                         | user_id, action, timestamp required                     |
| Default        | Users.status                 | Defaults to 'active'                                    |
| Default        | Folders.permissions          | Defaults to 'public'                                    |
| Default        | Documents.download_count     | Defaults to 0                                           |
| Default        | parent_folder_id             | Defaults to NULL (root level)                           |
| Cascade Delete | Folder deleted               | All documents and subfolders inside are deleted         |
| Cascade Delete | Document deleted             | All its permissions are deleted; logs remain (null ref) |
| Set Null       | User deleted                 | Folder created_by and document uploaded_by set to NULL  |

## 6. Indexing Strategy

Indexes are applied to columns that are queried or sorted frequently:

- Users.email — fast login lookups
- Foreign key columns (role\_id, folder\_id, uploaded\_by, parent\_folder\_id) — join performance
- Logs.timestamp — date-range filtering
- Logs.action — filter by action type
- Documents.download\_count — sort by popularity

---

## 7. Security & Storage Considerations

### Password Storage

Passwords are stored as bcrypt hashes. The hash includes a salt, ensuring each hash is unique even for identical passwords. Plain-text passwords are never persisted.

### File Storage Separation

Only text-based metadata and file paths are stored in the database. Binary file content lives on the file system. This keeps the database compact and simplifies backups.

### Account Safety

User accounts are deactivated via the 'status' field rather than deleted, preserving data history and referential integrity across folders, documents, and logs.

### Audit Trail Permanence

Log entries are never deleted. When a document is deleted, its log records remain with a null document reference, ensuring a complete and tamper-evident activity history.

---

## 8. Scalability Design

- Normalized schema prevents duplicate data across all tables
- Indexed foreign keys ensure efficient multi-table joins at scale
- File-database separation keeps database size manageable
- Extensible Roles table for adding new user types without schema changes
- Flexible Document Permissions for granular access without redundancy
- Self-referential Folders support unlimited nesting depth
- Denormalized download\_count avoids expensive COUNT aggregations on the Logs table